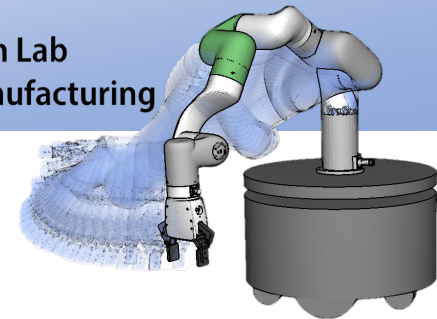


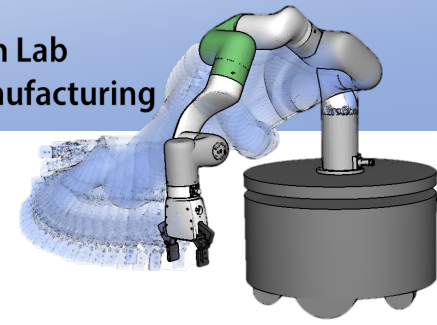


C言語：ポインタ・文字 列・構造体・ファイル

原田研究室 准教授 万 偉偉

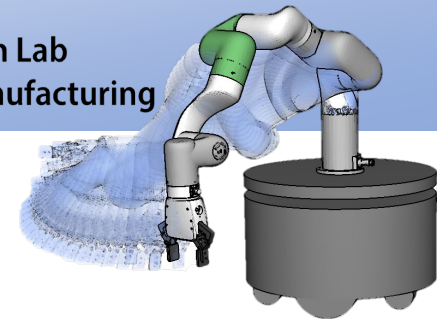
基礎工学部 F522号室

wan@sys.es.osaka-u.ac.jp



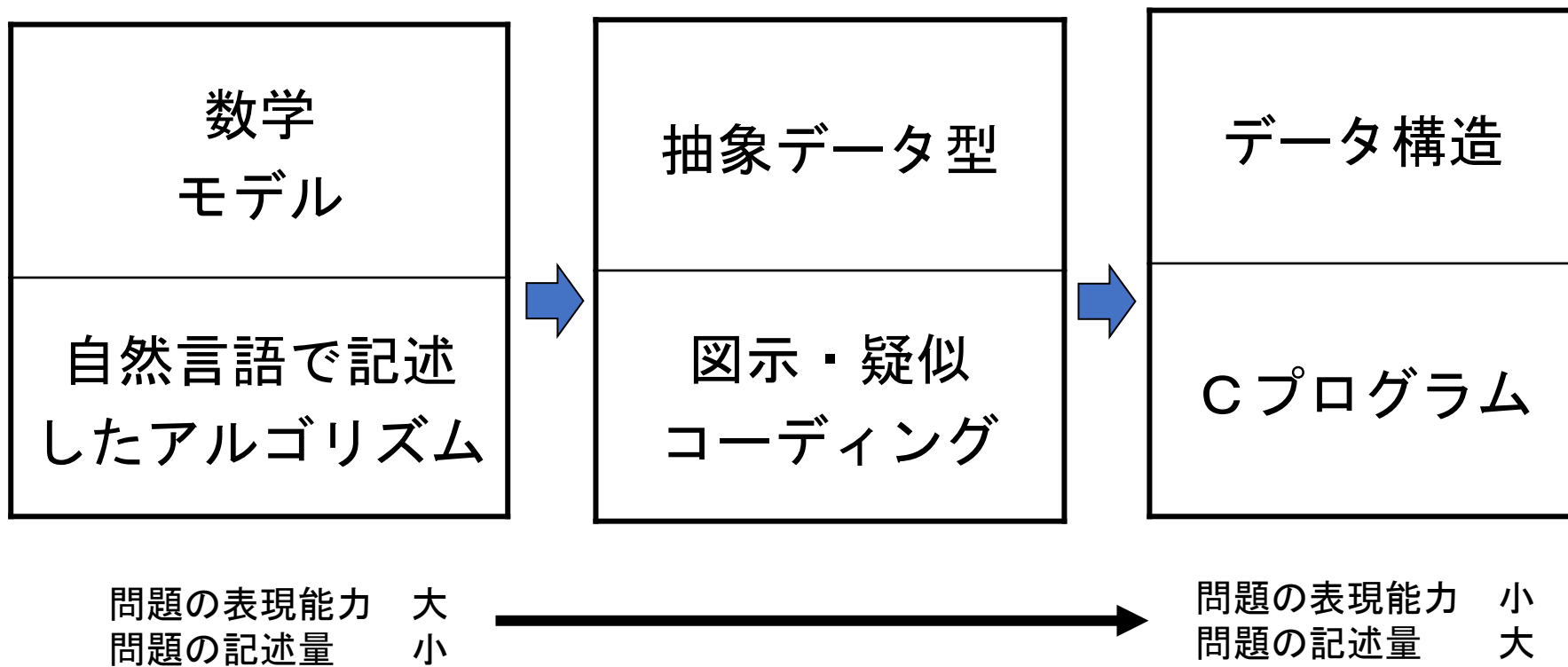
復習（今までの内容）

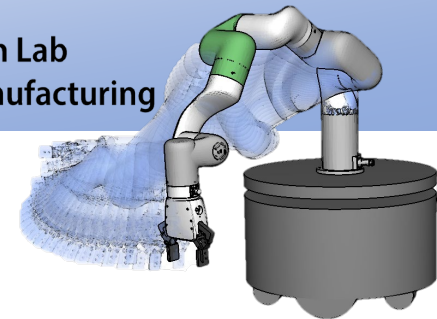
- アルゴリズム
 - 問題を解くための手順
- プログラミング
 - データ構造の選択とアルゴリズムの考慮
- データ型・データ構造
 - 整数・実数・配列・ポインタ等
 - リスト、待ち行列・スタック・木・グラフ等
- アルゴリズムの図示
 - フローチャート・NSチャート
- 疑似言語
 - BNF定義



復習（今までの内容）

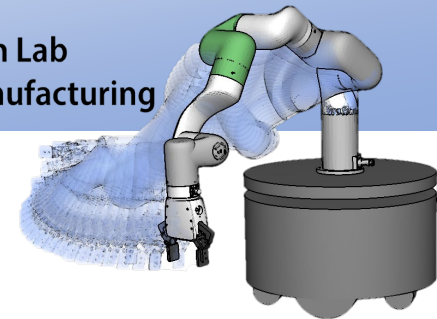
・ 問題解決の過程





復習（今までの内容）

- 配列
 - 宣言、初期化（2次配列、3次配列）
 - 最大値の計算
 - 2次配列の横計、縦計、総合計の計算
- 関数
 - 宣言、仮引数、実引数、呼び出し方法、返り値
 - 局所変数、大域変数
 - 記憶クラス：
 - static 隠す、初期化、メモリ確保
 - auto 大域変数に付けない
 - extern リンケージ：複数のファイル
 - 関数間のデータのやり取り
 - 値渡し、参照渡し、大域変数を利用、ポインタの利用



extern指定子

- extern指定子

- 分割コンパイルをする場合で、一つの大域変数を複数の翻訳単位で共通に用いる.

main.c

定義

```
#include <stdio.h>

int x=5; ← 同じメモリ領域を参照

void modify(void);

int main(void)
{
    printf("original: x=%d\n", x);
    modify();
    printf("modified: x=%d\n", x);
    return 0;
}
```

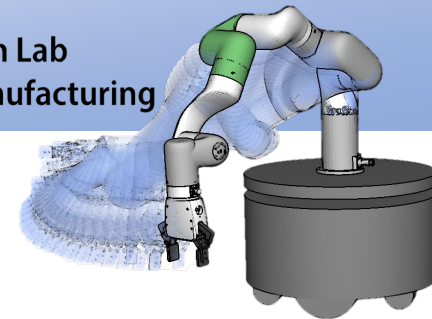
modify.c

```
extern int x;

void modify(void)
{
    x=x*2;
}
```

宣言のみ

同じ変数名の定義を2回以上行うのはおかしいため



staticを付けたら

- 見えなくなる

main.c

```
#include <stdio.h>

static int x = 5;

void modify(void);

int main(void)
{
    printf("original: x=%d\n", x);
    modify();
    printf("modified: x=%d\n", x);
    return 0;
}
```

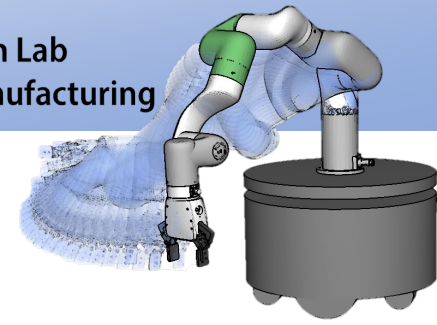
modify.c

```
extern int x;

void modify(void)
{
    x = x * 2;
}
```

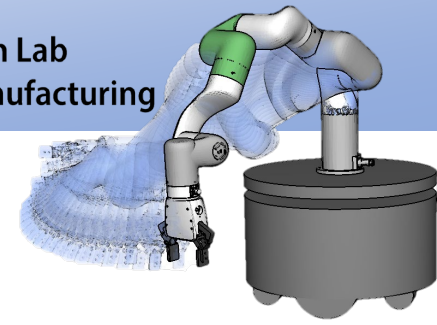
← 同じメモリ領域を参照

見えなくなる

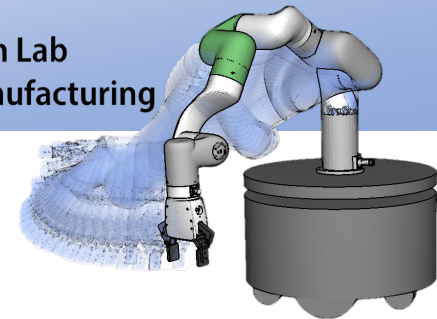


今日の内容

- C言語の“高級”データ型
 - ポインタ・文字列・構造体・ファイル



C言語：ポインタ



左辺値と右辺値

- 変数には2つの値がある
- 左辺値(left value)
- 右辺値(right value)

右辺値-> 8,.1

-> i, counter

中身・実際の値

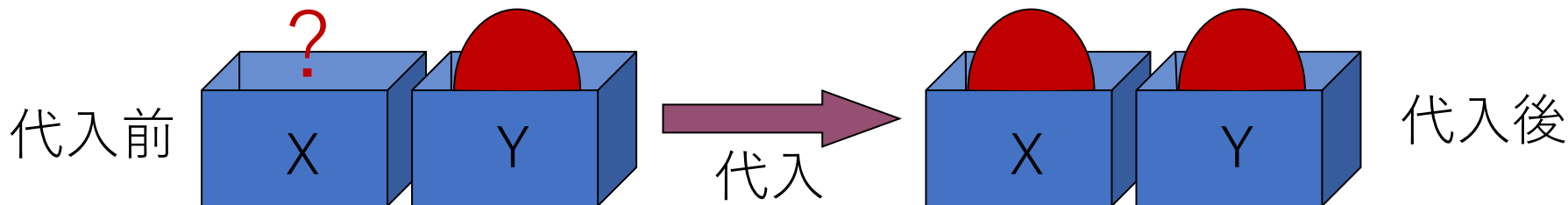
左辺値->変数だけ

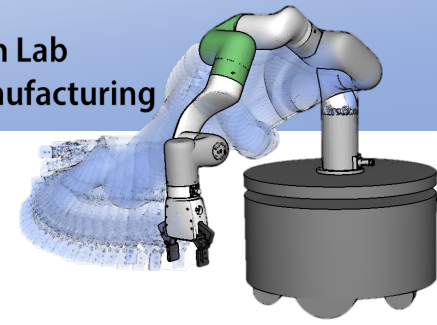
->場所

代入文 : $X \leftarrow Y$

(格納場所を表す) 左辺値

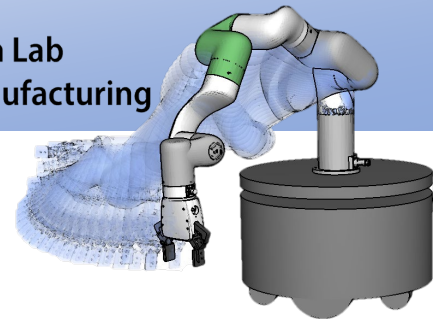
右辺値 (内容を表す)





左辺値と右辺値

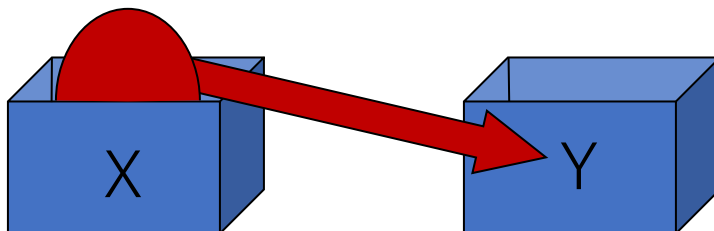
- 右辺値(right value)
 - 変数の値
- 左辺値(left value)
 - 変数の値が格納されている記憶装置の位置
 - 有効アドレス, Effective Address



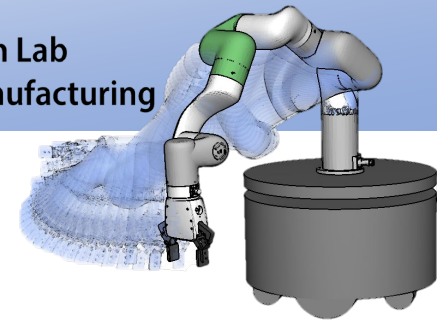
ポインタ (pointer)

- 計算機アドレスの抽象化概念
 - データ型
 - 有効アドレス
 - データサイズ
- 変数の左辺値を右辺値として持つ変数
- つまり、記憶装置の位置を内容とする変数

例：XがYをさすポインタ変数

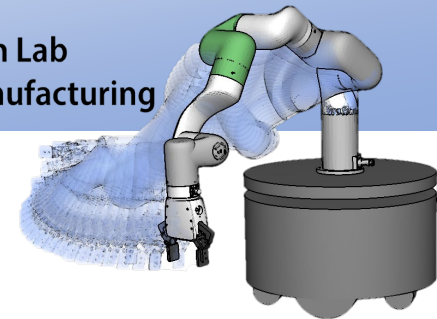


Yの記憶装置の
位置を指している



ポインタの演算

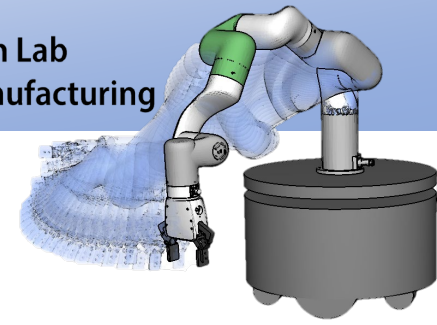
- 代入 (Substitution)
- 参照 (Reference)
 - 指している位置の中身を獲得すること
- 前進 (インクリメント, Increment)
- 後進 (デクリメント, Decrement)
- 等価 (同一のものを参照しているか?)



演算子から見たポインタと 計算機アドレスの違い

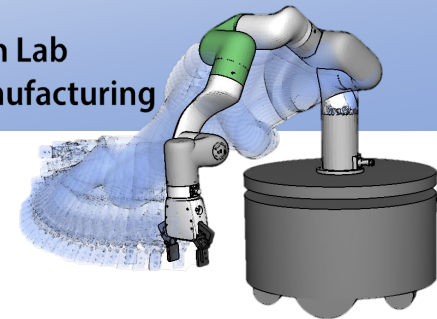
- 代入
 - ポインタの場合
 - 型が違うと代入できない
 - 計算機アドレスの場合
 - なんでも代入可能（機械語）
- 参照
 - ポインタの場合
 - 参照されたデータは型で解釈される
 - 計算機アドレスの場合
 - なんでも可能（機械語）

機械語での
アドレスのこと
C言語はコンパイ
ラーに制限される



演算子から見たポインタと 計算機アドレスの違い

- 前進, 後進(increment, decrement)
 - ポインタの場合
 - 型から決定されるデータサイズだけ増加/減少する
 - 計算機アドレスの場合
 - 機械語に依存する (1 or 2ワード前進/後進する)
- ポインタは計算機アドレスと異なり, 強く型に縛られている



C 言語のポインタ

• ポインタ 宣言

int *apnt; (アスタリスク ”*” をつける)

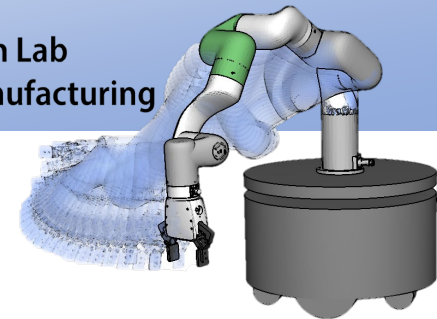
右辺値の型を示す

例) int xadd = 1234;
int *apnt;
apnt = &xadd;

apnt 左辺値を格納する領域

xadd 1234

演算子&は左辺値を返す



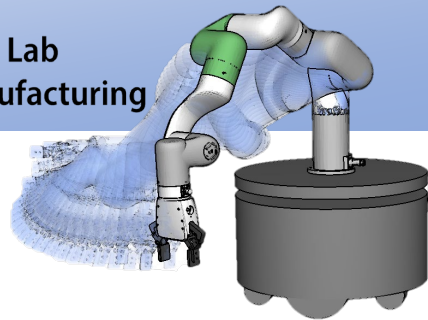
C言語のポインタ

- 参照

- ポインタがさすデータ領域の値を取り出す操作

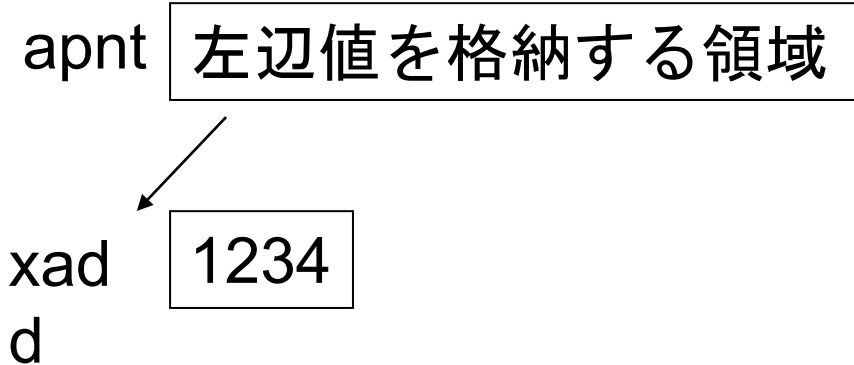
*ポインタ変数名

演算子*によりポインタ参照が行われる



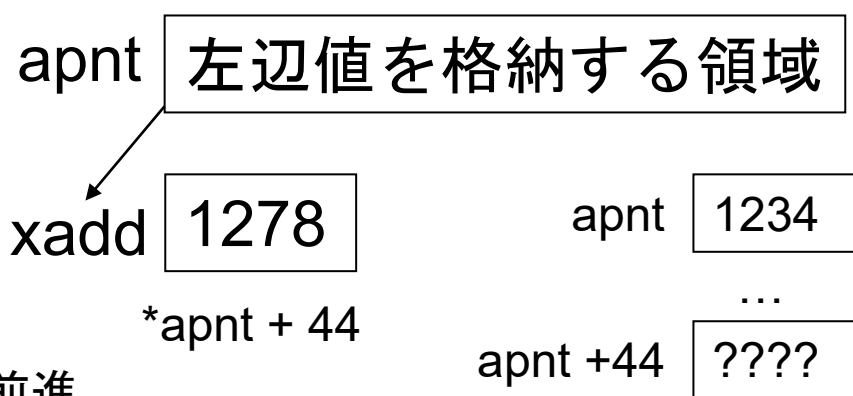
C言語のポインタ

例) `int xadd = 1234;`
`int *apnt;`
`apnt = &xadd;`

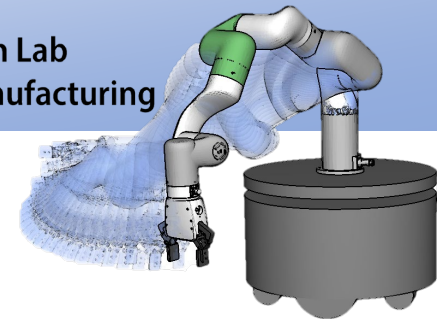


例) `*apnt = *apnt + 44;`

apnt が指すデータ領域に加算

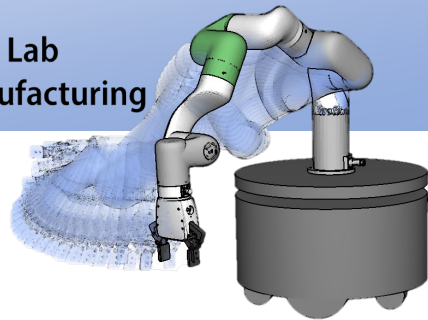


注) `apnt = apnt + 44;` との違い
 44個のint分のデータサイズだけ前進



C 言語の配列の謎

配列を関数に引き渡して関数内で値を書き換えると、なぜ関数を出てもその変更が維持されるのか？



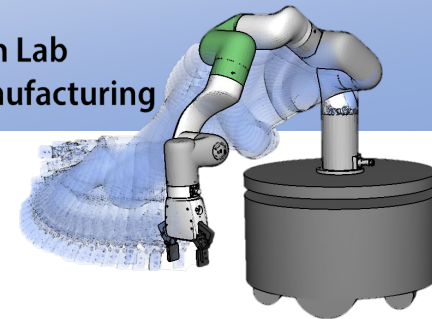
C言語のポインタと配列

- 配列名は、ポインタ型の右辺値をもつ
- 左辺値は持っていないので、配列名へ代入はできない
- 配列名は、その一連の領域の先頭アドレスを指したポインタ、0番を基点として相対アドレスとしても考えられる

例) char astr[10];

	astr+0	+1	+2	+3	+4	+5	+6	+7	+8	+9
astr	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

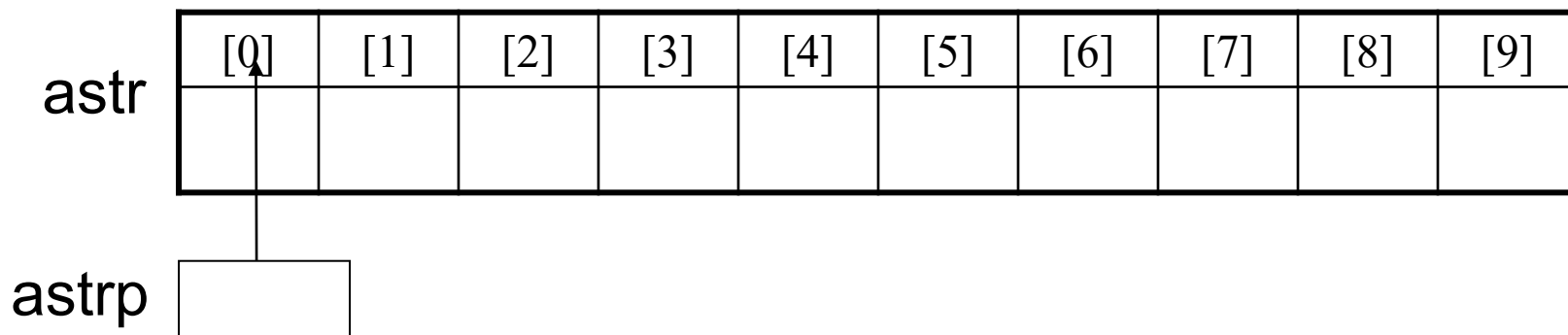
データの1番 astr[1] は astr+1 の位置に格納 *(astr+1)

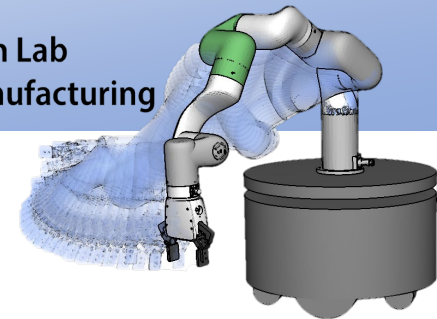


ポインタの前進, 後進 ポインタの加算, 減算 (C言語)

- 整数型との加減算が可能
- 前進/後進するデータ数を整数型が指定する
($p+j-1$, $p++$)

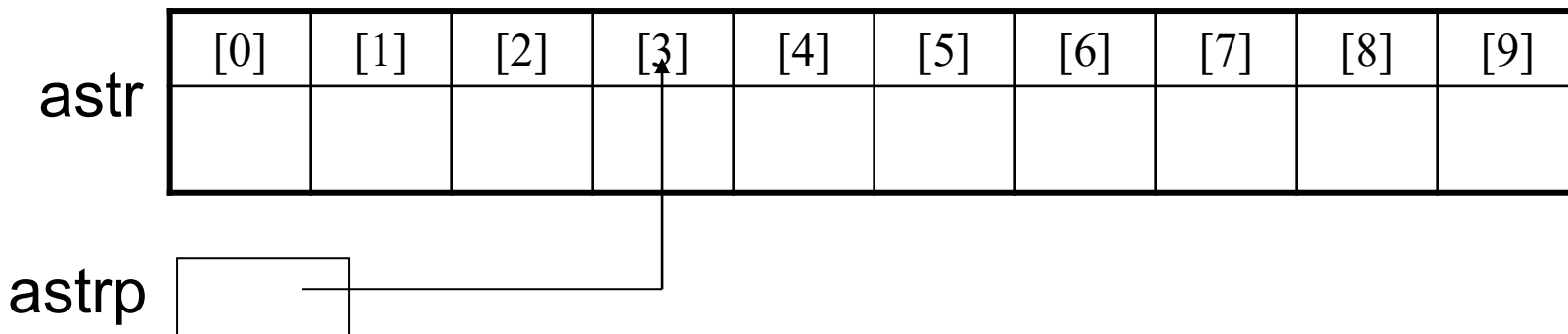
例) `char astr[10];`
 `char *astrp;`
 `astrp = astr;`

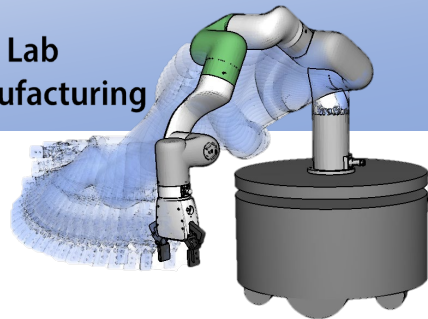




ポインタの前進, 後進 ポインタの加算, 減算 (C言語)

例) `char astr[10];`
 `char *astrp;`
 `astrp = astr;`
 `astrp += 3;`





ポインタの前進, 後進 ポインタの加算, 減算 (C言語)

- 加減算されるバイト数はポインタが指すデータ型のサイズで決まる

short sdata[4]; sdata: 0x1000 short *sp=sdata;

short ->2byte

[0]	[1]	[2]	[3]

char cdata[8]; cdata: 0x1000 char *sp=cdata;

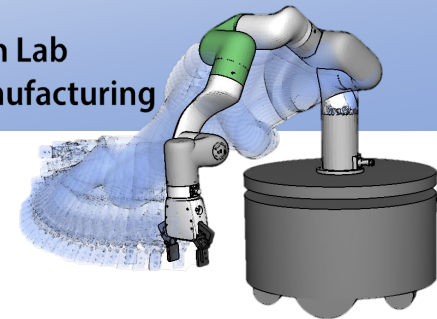
char->1byte

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

sp+=2; → **0x1004**

cp+=2; → **0x1002**

共に記憶空間は ? バイト消費する



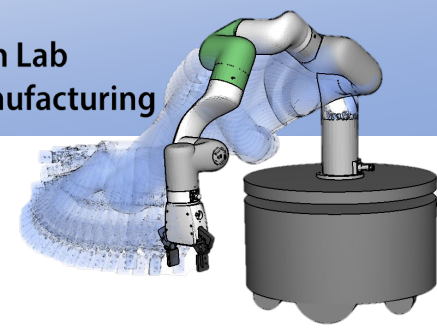
サンプルプログラム

```
#include <stdio.h>
int main(void)
{
    char str[8]; char *strp = str;
    short sdata[4]; short *sp = sdata;

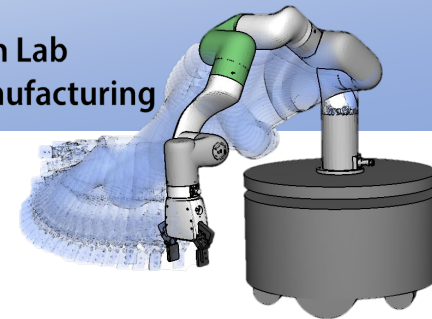
    printf( "datasize str = %d, sdata = %d¥n", sizeof(str), sizeof(sdata) );
    printf( "str = %08X, sdata = %08X¥n", str, sdata );
    printf( "strp = %08X, sp = %08X¥n", strp++, sp++ );
    printf( "strp = %08X, sp = %08X¥n", strp++, sp++ );
    return 0;
}
```

↑ 16進数表現で出力する変換記号(X)

実行例) datasize str = 8, sdata = 8
 str = BFFFF170, sdata = BFFFF180
 strp = BFFFF170, sp = BFFFF180
 strp = BFFFF171, sp = BFFFF182

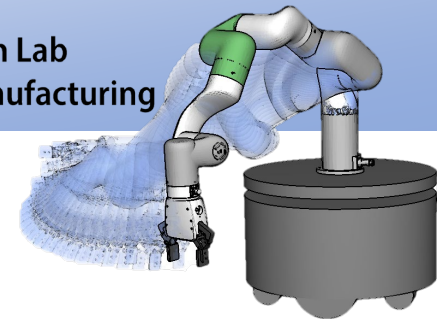


C言語：文字列



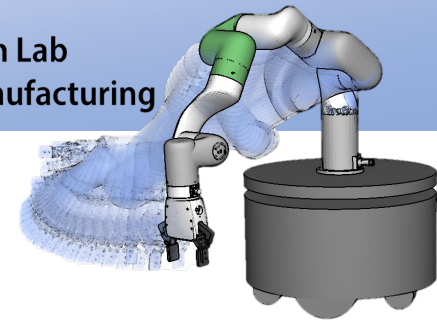
文字型

- 文字型(character)
 - 1 バイト(256文字)が格納できる ASCII
 - 日本語の文字（約65,000字以上といわれる）を表現するには2バイト以上必要
 - 文字型というが、実は日本語の文字を格納するには記憶領域が足りない
 - 計算機、言語、概念が西欧言語(たかだか20数文字)諸国で開発されているため
 - 実際には単に1バイトを表す型の方が正確
 - 特に英語向け



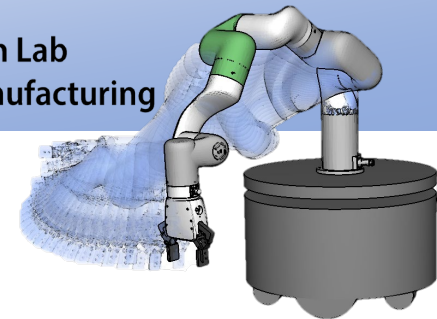
文字列(String)

- 文字列型はC言語にはない
- 文字列は文字型の並び（1次元配列）として表現する
- 文字列の末尾には終端記号¥0が必要
- 抽象データ型ADT Abstract Data Type
 - ADT String の操作はライブラリで提供
 - コピー，代入，連結，部分文字列，比較



ADT String

- コピー, 代入
- 連結
 - “abc”+ “def” $\rightarrow$ “abcdef”
- 部分文字列
 - `substring( “abcdef”, 2, 3 )  $\rightarrow$  “bcd”`
- 比較
 - 辞書式順序
 - “a” < “aa” < “ab” < ... < “z” < ...



C 言語の文字列のコピー

- `char *strncpy( char *dst, char *src, size_t n );`

```
#include <stdio.h>
```

```
#include <string.h> /* 文字列ライブラリを宣言したヘッダファイル */
```

```
int main(void) {
```

```
    char src[] = "ABCDEFGH"; /* 複写元の領域 */
```

```
    char dst[10]; /* 複写先の領域(コピーできるだけの領域要) */
```

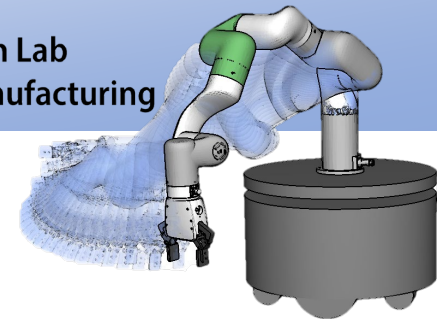
```
    char *p; /* 返り値のポインタ, dst と同じ */
```

```
    p = strncpy(dst, src, sizeof(src) ); /* 文字のコピー */
```

```
    printf( "p = %s, dst = %s¥n", p , dst );
```

```
    return 0;
```

```
}
```



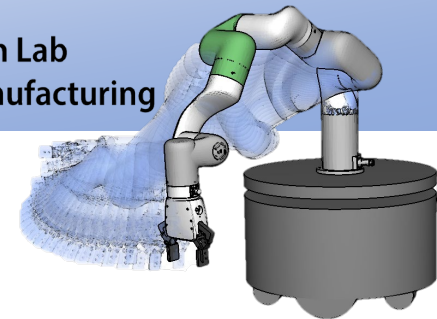
C言語の文字列のコピー

```
char src[] = "ABCDEFGG";
char dst[10];
```

src	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	A	B	C	D	E	F	G	¥0

文字列の終わりを示す終端記号

[illegible]

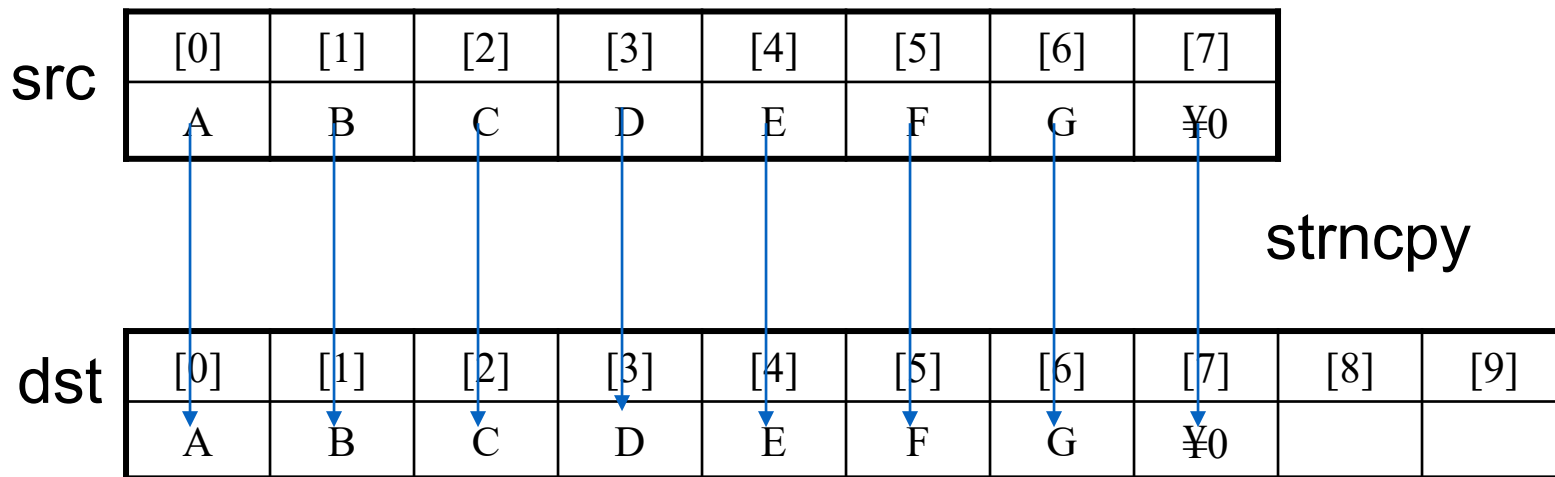


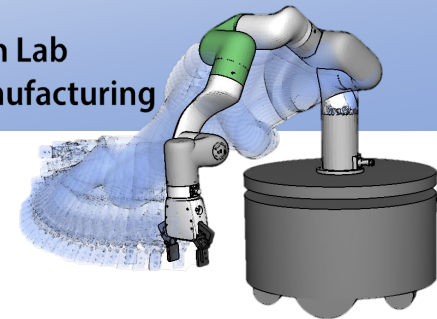
C 言語の文字列のコピー

```

char  src[] = "ABCDEFGG";
char  dst[10];
p = strncpy( dst, src, sizeof(src) );
  
```

コピーするバイト数 = 8





C 言語の文字列の長さ

- strlen関数を利用する

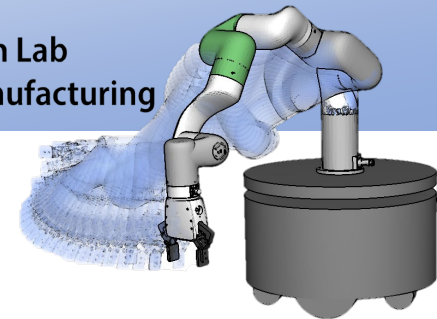
```
#include <stdio.h>
#include <string.h>
```

```
int main(void) {
    char src[] = "ABCDEFGH";
    printf( "len = %d, size = %d\n", strlen(src), sizeof(src) );
    return 0;
}
```

実行例)

len = 7, size = 8

strlen関数は終端記号を含めないで長さを計算する



C 言語の文字列操の連結

- 連結(concatenate)

- `char *strncat( char *s, char *scat, size_t n );`
- 文字列 `s` の終わりに `scat` を `n` バイト分だけ連結する.
- 文字ポインタ `s` が指している領域は, `strlen(s)+n` より大きくなければならない

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char src[10] = "foo";
    char cat[] = "bar";
    char *p;
    p = strncat( src, cat, strlen(cat) );
    printf( "cat = %s¥n", p );
    return 0;
}
```

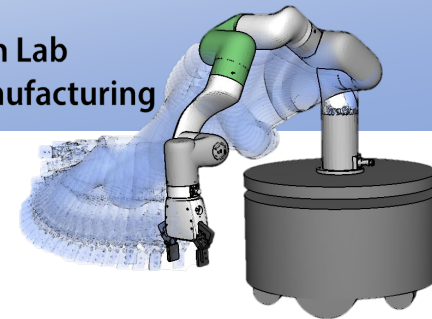
文字列を `n` 文字連結

【書式】

```
#include <string.h>
char *strncat(char *s1, const char *s2, size_t n);
```

【説明】

文字型配列 `s1` のうしろに文字列 `s2` を `n` 文字分連結します。
`s2` の長さが `n` 以上のときには `n` 文字を連結し `'¥0'` を付加します。`s2` の長さが `n` より少ない場合には `'¥0'` まで連結します。(下図参照)
`s1` を超して連結した場合と、`s1` と `s2` が重なっていた場合は、動作は未定義となります。

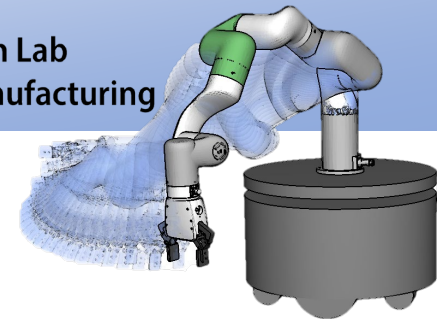


C 言語の文字列操の連結

```
char src[10] = "foo";  
char cat[] = "bar";
```

cat	[0]	[1]	[2]	[3]
	b	a	r	¥0

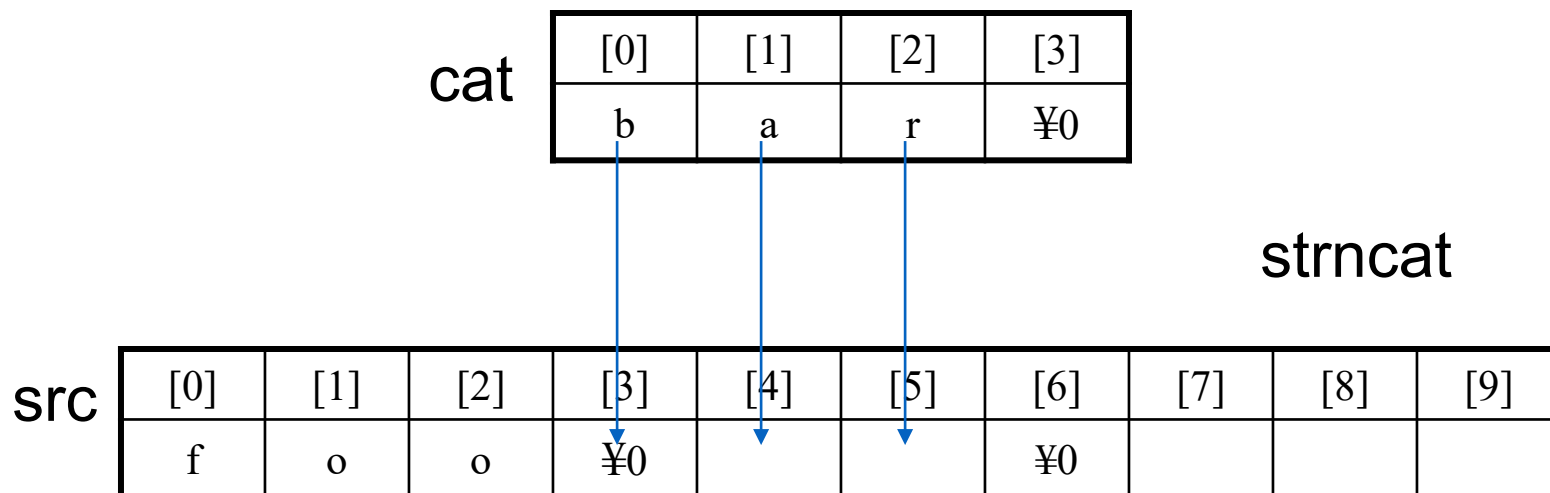
src	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	f	o	o	¥0						

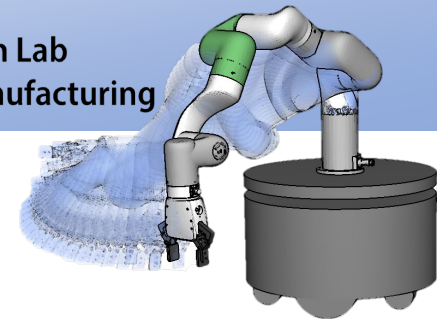


C 言語の文字列操の連結

```

char src[10] = "foo";
char cat[] = "bar";
strncat( src, cat, strlen(cat) );
  
```





C 言語の文字列操の連結

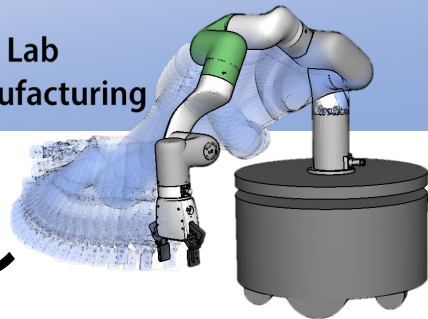
```
char src[10] = "foo";  
char cat[] = "bar";  
strncat( src, cat, strlen(cat) );
```

cat

[0]	[1]	[2]	[3]
b	a	r	¥0

src

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
f	o	o	b	a	r	¥0			



C 言語の部分文字列の取り出し

• ポインタとコピー

```
#include <stdio.h>
#include <string.h>
```

```
strncpy(copygoal, copysource, bytesize)
```

```
int main(void) {
    char src[] = "foobar"; /* コピー元 */
    char dst[4]; /* コピー先 */
    strncpy(dst, &src[3], 3 ); /* srcの4バイト目から3バイト分コピー */
    dst[3] = '¥0'; /* 最後に終端記号を代入 */
    printf( "dst = %s¥n", dst );
    return 0;
}
```

【書式】

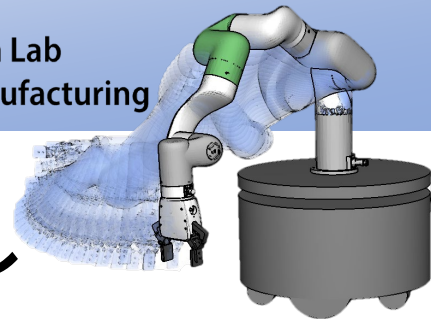
```
#include <string.h>
char *strncpy(char *s1, const char *s2, size_t n);
```

【説明】

文字型配列 *s1 に文字列 *s2 を先頭から n 文字コピーします。

① s2 の長さが n 以上のときには n 文字をコピーしますが、このときに '¥0' の自動付加は行わないので注意が必要です。

② s2 の長さが n より少ない場合には残りの文字を '¥0' で埋めます。
もし、s1 と s2 が重なっている場合には動作は未定義となります。

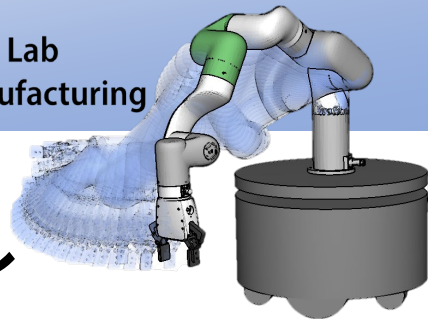


C言語の部分文字列の取り出し

```
char src[] = "foobar";  
char dst[4];
```

src	[0]	[1]	[2]	[3]	[4]	[5]	[6]
	f	o	o	b	a	r	¥0

dst	[0]	[1]	[2]	[3]



C 言語の部分文字列の取り出し

```

char src[] = "foobar";
char dst[4];
strncpy( dst, &src[3], 3 );
  
```

src[3]の左辺値が必要なので

&演算子が必要.

そうでなければ右辺値の'b'が渡されてしまう

&src[3]

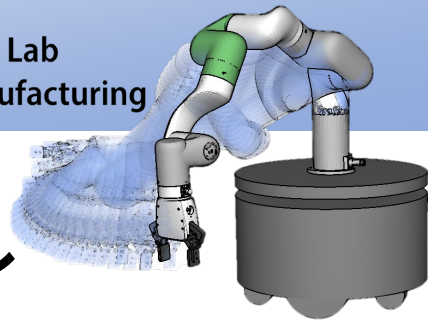
src

[0]	[1]	[2]	[3]	[4]	[5]	[6]
f	o	o	b	a	r	¥0

dst

[0]	[1]	[2]	[3]
b	a	r	

strncpy



C 言語の部分文字列の取り出し

```
char src[] = "foobar";
char dst[4];
strncpy( dst, &src[3], 3 );
dst[3] = '¥0';
```

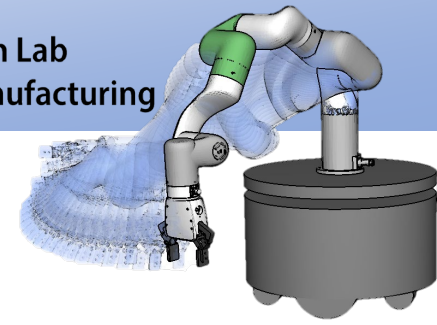
src

[0]	[1]	[2]	[3]	[4]	[5]	[6]
f	o	o	b	a	r	¥0

dst

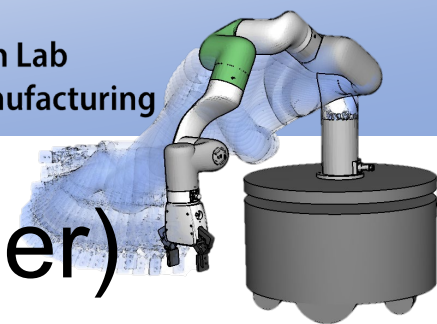
[0]	[1]	[2]	[3]
b	a	r	¥0

文字列の終端記号 '¥0' を末尾に代入



C 言語の文字列の比較

- 比較(compare)
 - `int strcmp( char *s1, char *s2);`
 - 文字列 `s1`と文字列`s2`を辞書式順序で比較する. 等しければ 0, `s1<s2`の場合は 0 より小さい値が, `s1>s2`の場合は 0 より大きい値が返る



辞書式順序(lexicographical order)

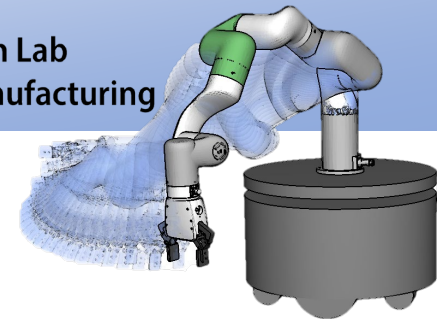
- 各文字は比較可能であること.
- 空文字 ε は他の全ての文字より小さい $\forall \alpha [\varepsilon < \alpha]$
- 比較する文字列同士の長さが異なるときは空文字 ε を連結して長さを揃える

$$S1 = \alpha_1 \alpha_2 \cdots \alpha_N, S2 = \beta_1 \beta_2 \cdots \beta_N$$

$$S1 < S2 \Rightarrow \forall k [\alpha_k < \beta_k] \text{ または,}$$

$$\exists i [\alpha_n = \beta_n, \alpha_{i+1} < \beta_{i+1} \mid n = 1 \cdots i]$$

例) “aa” < “aaa”, “ab” < “abc”, “aaa” < “ab”



例題 文字列の反転

- 単語をキーボードから入力してはそれを反転して出力する. “quit” が入力されたときに終了する. なお, 文字列の反転は, 関数reverse を定義し, その関数の中で行う.

<アルゴリズムの考え方>

2つの関数 main, reverse を作成 (仕様より)

main関数で, 入力文字列と反転結果文字列の格納領域を確保する (最大入力文字数N)

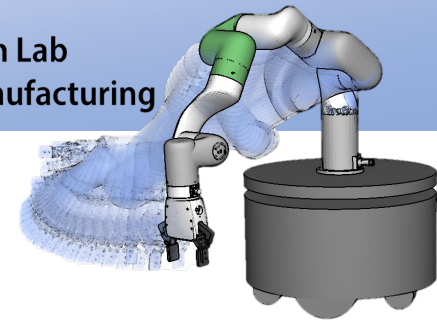
main関数 : 1) 単語 word をキーボードから入力する

2) word が “quit” でない限り, 以下を繰り返す

2-1) 関数reverseにより, word を反転する

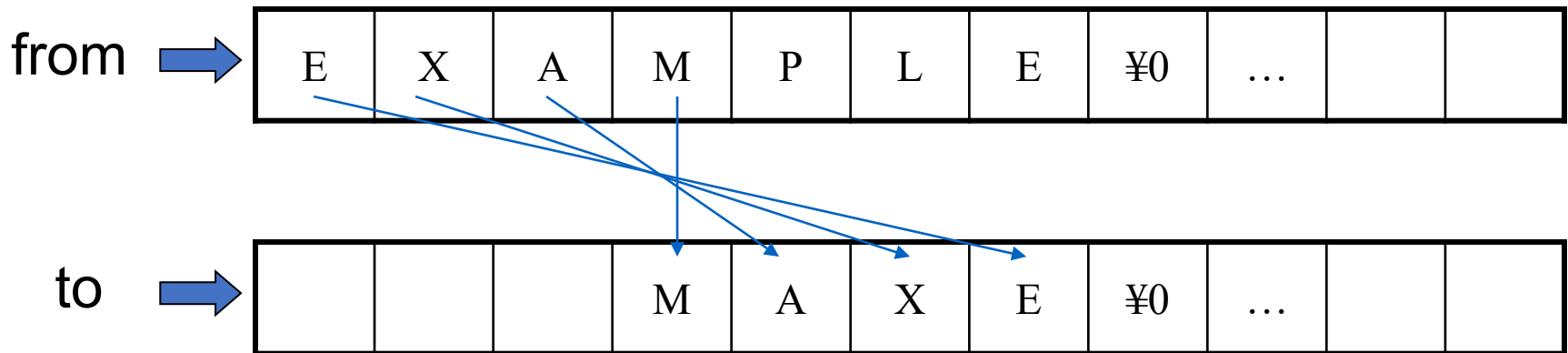
2-2) 反転結果を出力する

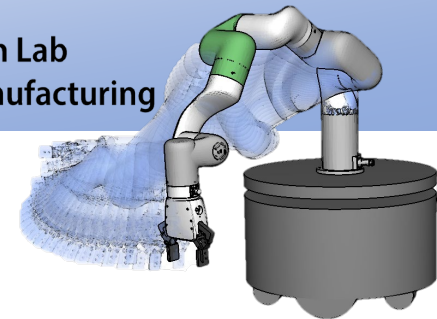
2-3) 次の単語 word を入力する



例題 文字列の反転

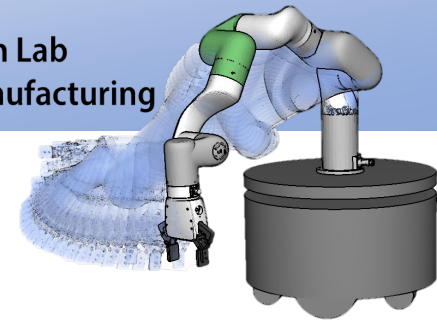
```
char *reverse( char *dst, char *src);
```





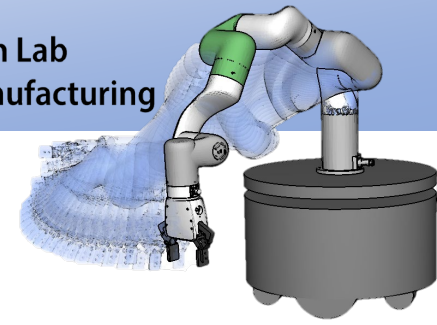
例題 文字列の反転

```
#include <stdio.h>
#include <string.h>
#define N 30
char *reverse(char *dst, char *src);
int main(void) {
    char word[N], result[N];
    printf( "word = " ); scanf( "%s", word );
    while ( strcmp( word, "quit" ) != 0 ) {
        printf( "reversed word = %s¥n", reverse(result, word));
        printf( "word = " ); scanf( "%s", word );
    }
    return 0;
}
```



例題 文字列の反転

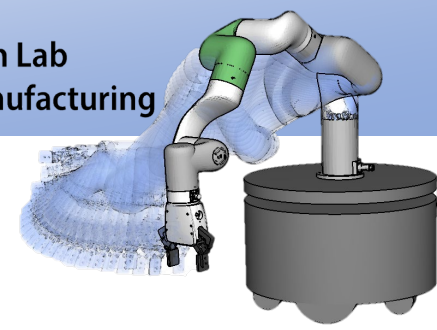
```
char *reverse(char *dst, char *src)
{
    dst += strlen(src);
    *dst = ¥0 ;
    while (*src != ' ¥0') {
        *--dst = *src++;
    }
    return dst;
}
```



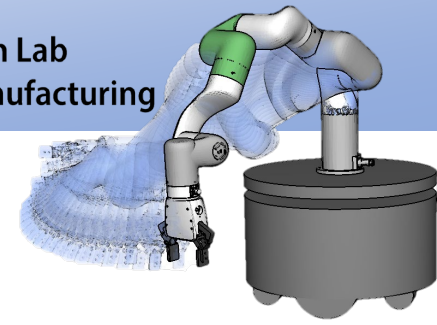
例題 文字列の反転(4)

```
char *reverse(char *dst, char *src)
{
    dst += strlen(src);
    *dst = '¥0';
    while (*src != '¥0') {
        *--dst = *src++;
    }
    return dst;
}
```

*と--
優先順位はどっ
ちが高い？



C言語：構造体

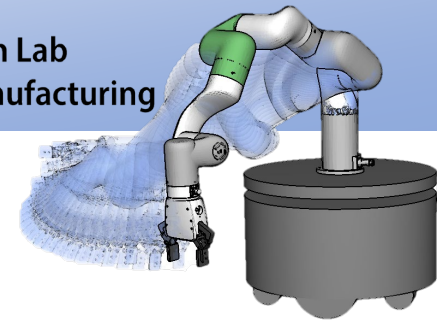


組(tuple)

- 直積集合

- 複数の集合の総当りを要素として持つ集合
- 同一の型でなくても良い.
- 例えば, 人名の集合 $\times$ 生年月日という空間上の点
(松本潤, 1983年8月30日)

個々のデータはある特定の空間上の点として表現できる

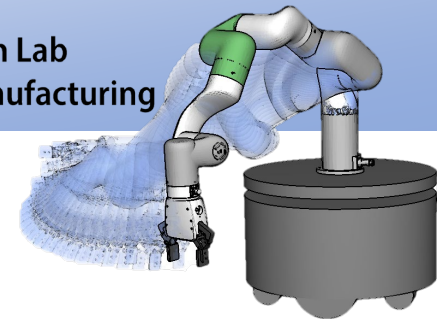


ADT レコード型(record)

- 組の概念に相当する
 - 組との違い
 - 組の要素にラベルと型がついている
 - 組の要素にラベルでアクセスできる

誕生日レコード ≡ (人名:文字列型, 生年月日:文字列型)

ラベル 型



構造体(structure)

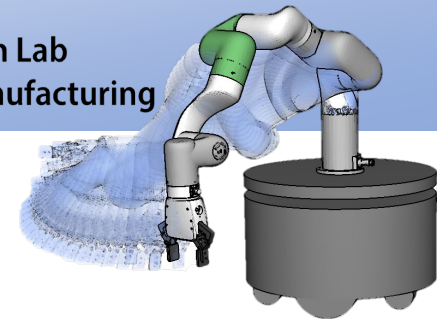
- C言語のデータ型のひとつ
 - ADT レコード型に相当する
 - 異なるデータ型を持つことができる
 - 構造体の構成要素を前もって定義する必要がある
 - それぞれの要素にはメンバ名（レコードではラベルに相当）でアクセスできる

○構造体の構成要素の定義

struct 構造体タグ名 {構造体宣言の並び} ;

↑型枠の名前

↑メンバ（構造体の構成要素）



構造体(structure)

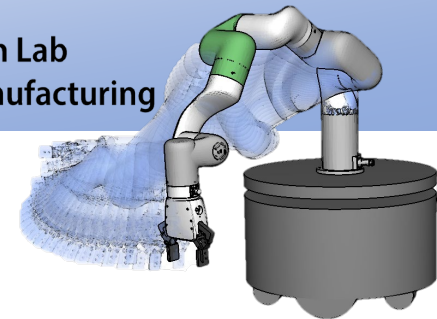
- 構造体の変数宣言

```
struct 構造体タグ名 変数名 ;
```

例)

```
struct SAMPLE { /* 構造体SAMPLEの定義開始 */  
    int  number; /* 整数型メンバ number */  
    char name[32]; /* 文字型配列メンバ name */  
}; /* 構造体定義終了 */
```

```
struct SAMPLE data;  
/* 構造体SAMPLE型変数 dataの宣言 */
```

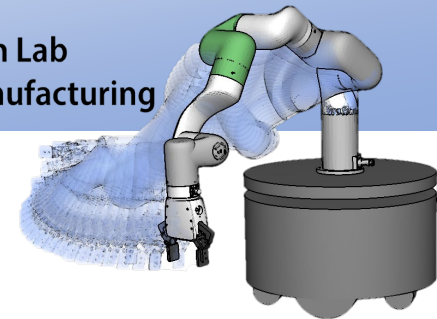


構造体(structure)

- 構造体のメンバへのアクセス
 - メンバアクセス演算子 (.)

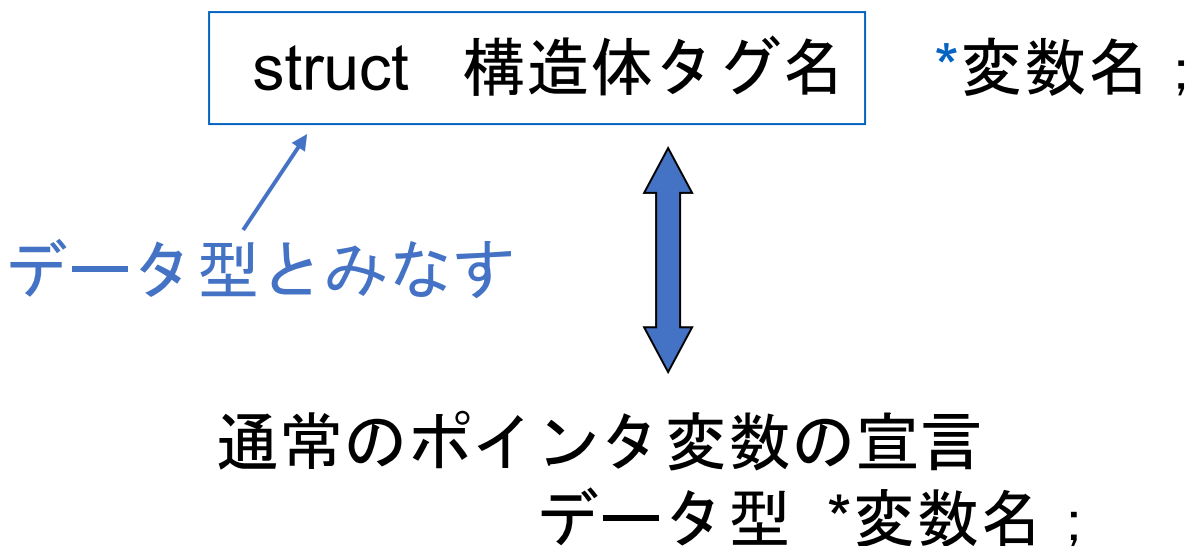
構造体名.メンバ名

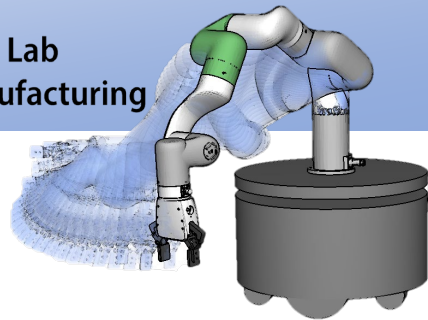
例) data.number data.name[0]



構造体へのポインタ変数

- 通常のポインタ変数と同様に宣言できる





構造体のメンバアクセス —ポインタ変数の場合—

例) struct SAMPLE *p;
 (*p).number (*p).name[0]

pが指す構造体

演算子優先順位 . → 1位 * → 2位

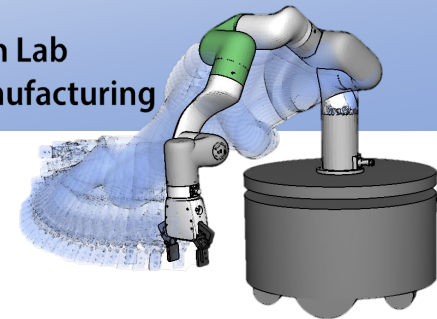
*p.number との違い

意味 : *(p.number)

構造体p（実際にはアドレスなので
無効）のメンバ numberの指す実体

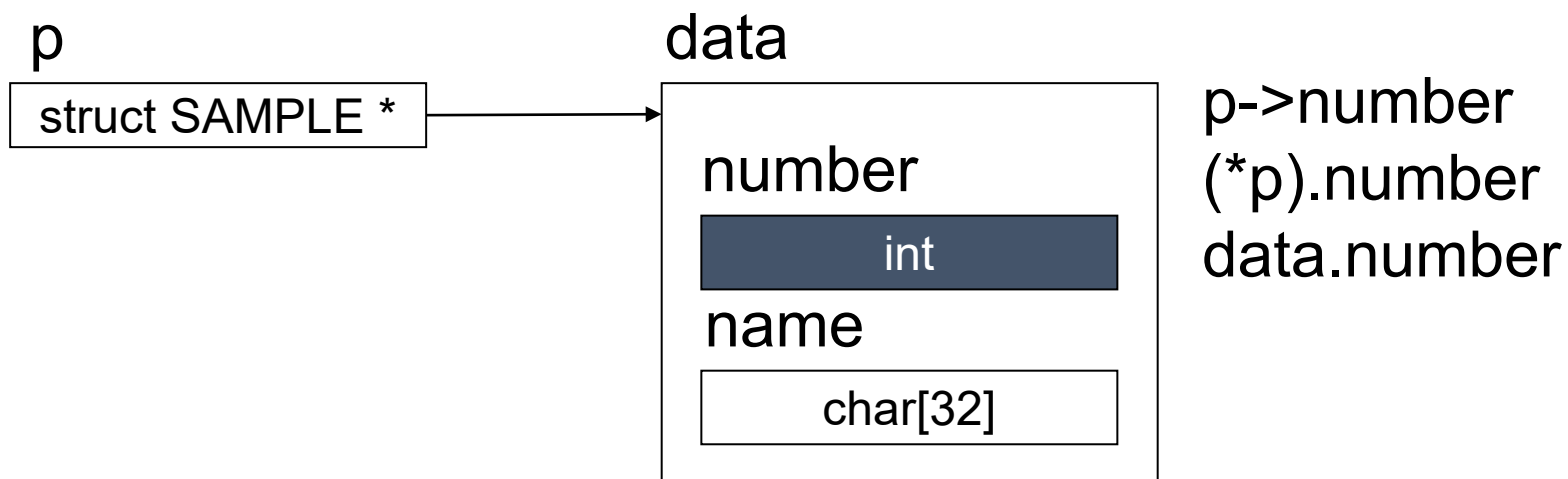
上と間違えやすいので別記法がある

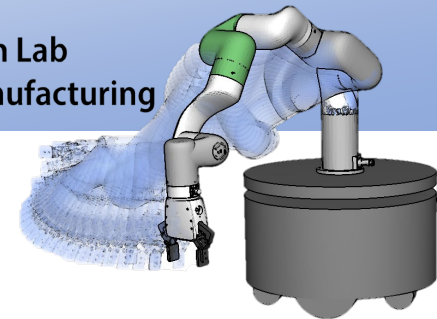
例) p->number p->name[0]



構造体メンバ参照例

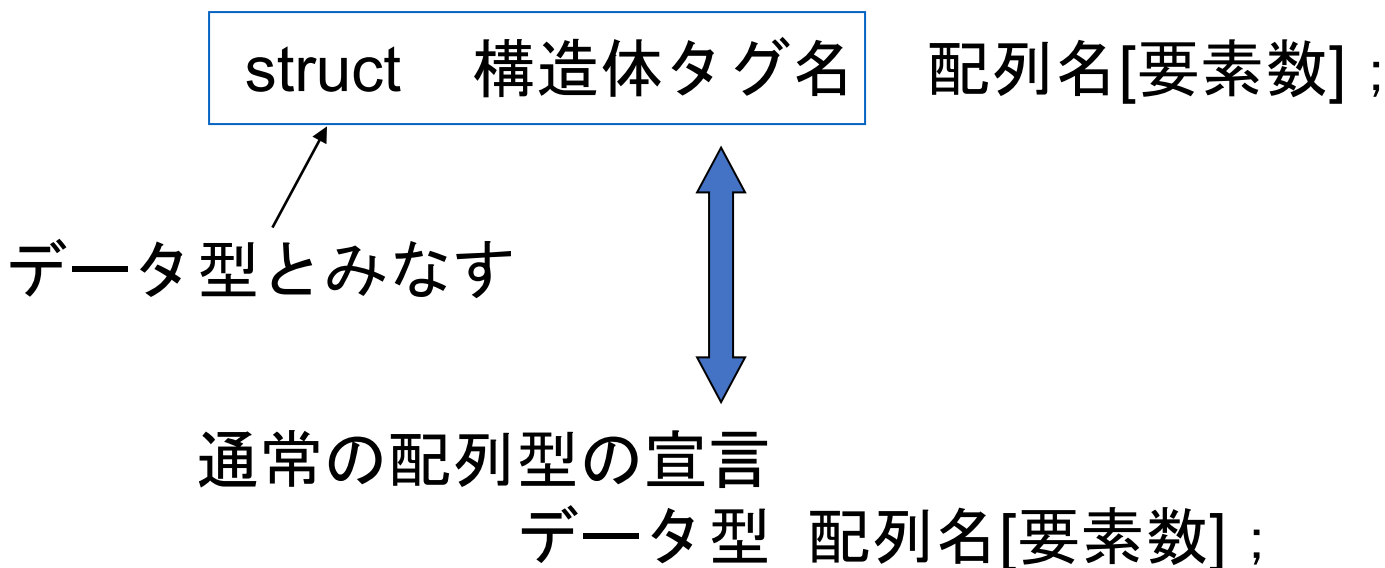
```
struct SAMPLE {  
    int number;  
    char name[32];  
};  
struct SAMPLE data, *p = &data;
```

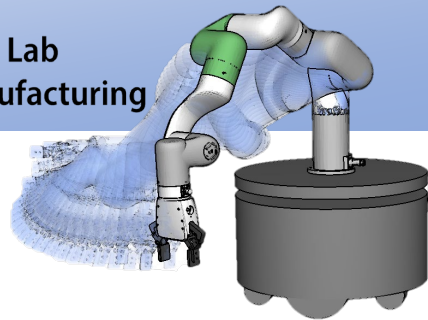




構造体の配列

- 基本データ型と同様に宣言できる



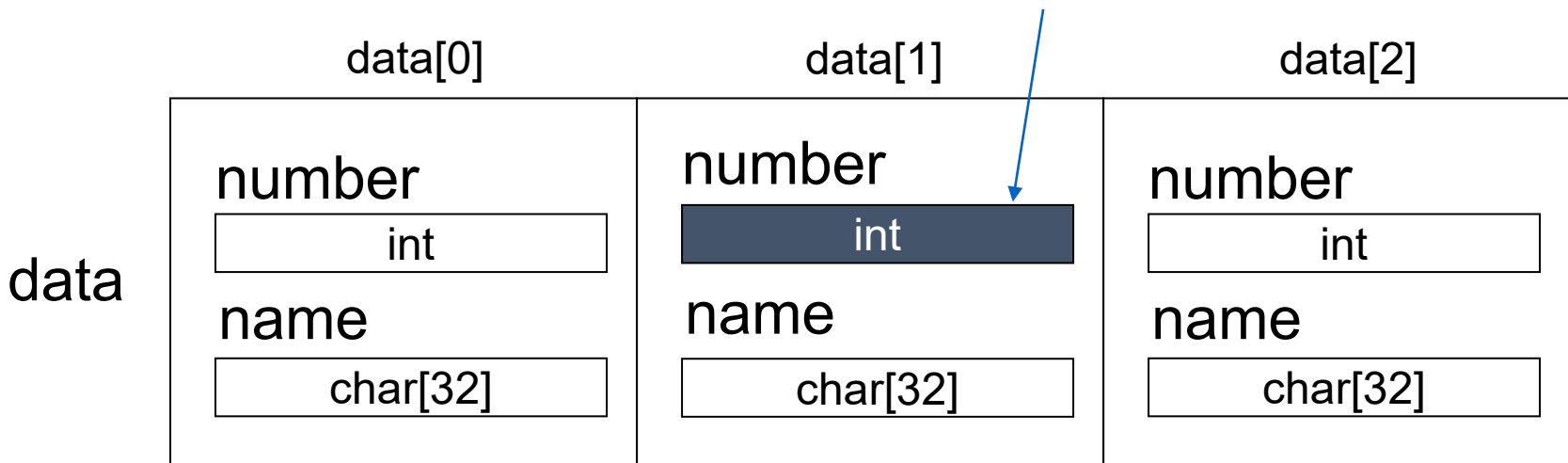


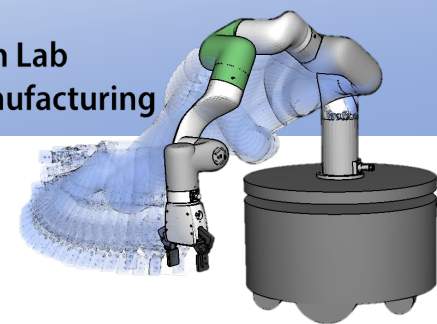
構造体の配列

```

struct SAMPLE {
    int number;
    char name[32];
};
struct SAMPLE data[3];
  
```

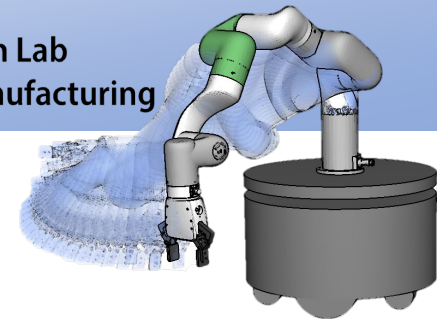
例) このメンバへのアクセス
 data[1].number





構造体の使用例

```
#include <stdio.h>
struct SAMPLE {
    int  code;
    char *name; /* ポインタ も構造体のメンバとして可能 */
    char phone[20]; /* 配列 */
};
int main(void) {
    struct SAMPLE adr[] = { /* 構造体配列の初期化 */
        { 1, "Tanaka", "0123-45-6789" },
        { 2, "Yukawa", "0123-56-7890" },
        { 3, "Koshiba", "0123-67-8901" } };
    int i;          /* ↓全体のサイズ/要素のサイズ=要素数 */
    for (i = 0; i<sizeof(adr)/sizeof(struct SAMPLE); ++i) {
        printf( "%02d [%-19s] Phone:%s¥n",
                adr[i].code, adr[i].name, adr[i].phone );
    }
    return 0;
}
```



struct SAMPLE の構造

```
struct SAMPLE {
    int  code;
    char *name;
    char phone[20];
};
```

sizeof(struct SAMPLE) = ?

struct SAMPLE

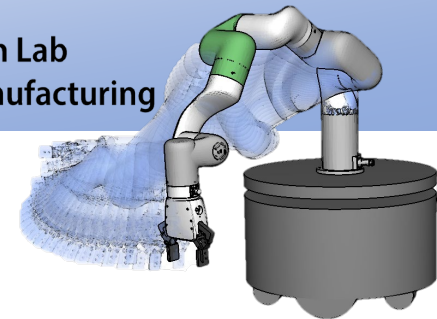
code	int
name	char *
phone	char[20]

int型 : 4バイトとする

ポインタ変数の
メモリ使用量



4バイトとする (32-bit compiler)



struct SAMPLE の構造

```
struct SAMPLE {
    int  code;
    char *name;
    char phone[20];
};
```

```
struct SAMPLE A = {
    3, "Yukawa",
    "0123-56-7890"
};
```

```
struct SAMPLE A;
->?
```

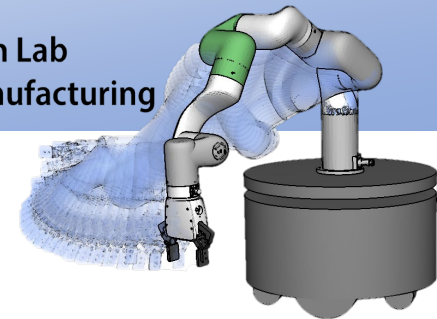
struct SAMPLE A

code	3
name	char *
phone	0123
	-56-
	7890
	'¥0'

初期値領域

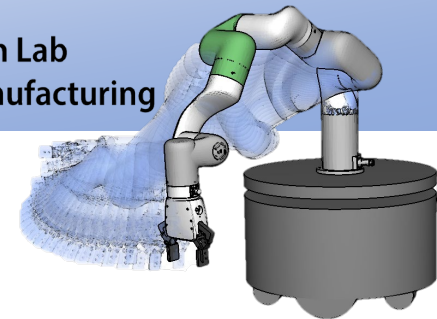
Yuka
wa'¥0'

注) 構造体メンバにポインタが含まれる場合、ポインタが指す実体は、コンパイラにより自動的に用意はされないの
で、プログラマが確保する必要がある。



メモリ領域の管理

- 静的変数（大域変数格納領域），自動変数（局所変数格納領域）のメモリ領域はコンパイラが管理している
- プログラム実行中に自分でメモリ領域を確保するには，ヒープ領域(heap)と言われるメモリ領域を管理するライブラリを利用する
- malloc関数
- free関数



メモリ領域の管理

malloc関数

外部仕様 : `void *malloc(size_t size)`

`size_t size; /* 確保したいバイト数 */`

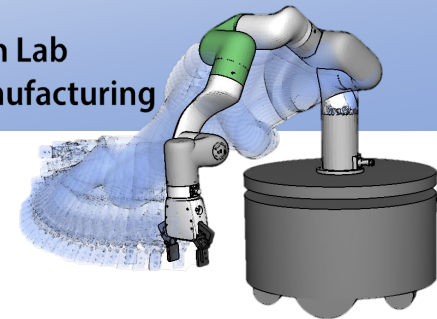
内部仕様 : `size` で指定したバイト数のメモリを確保する
確保された領域は 0 クリアはされない

返値 : メモリが確保できた場合, その領域の先頭
アドレスそうでない場合, `NULL` を返す.



C言語では実際の値は？

???



メモリ領域の管理(3)

- mallocを用いて, struct SAMPLE型のデータ領域を取得する場合

```
struct SAMPLE {  
    int  code;  
    char *name;  
    char phone[20];  
};
```

```
struct SAMPLE *p;
```

↓ struct SAMPLE型のサイズ計算

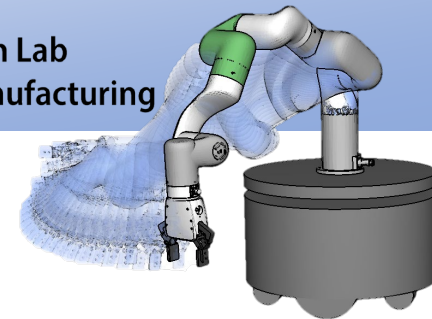
```
p = (struct SAMPLE *)malloc(sizeof(struct SAMPLE));
```

↑ struct SAMPLEポインタ型へ型変換

```
p->code = 3;
```

```
p->name = strdup( "Yukawa" );
```

```
strcpy( p->phone, "0123-56-7890" );
```

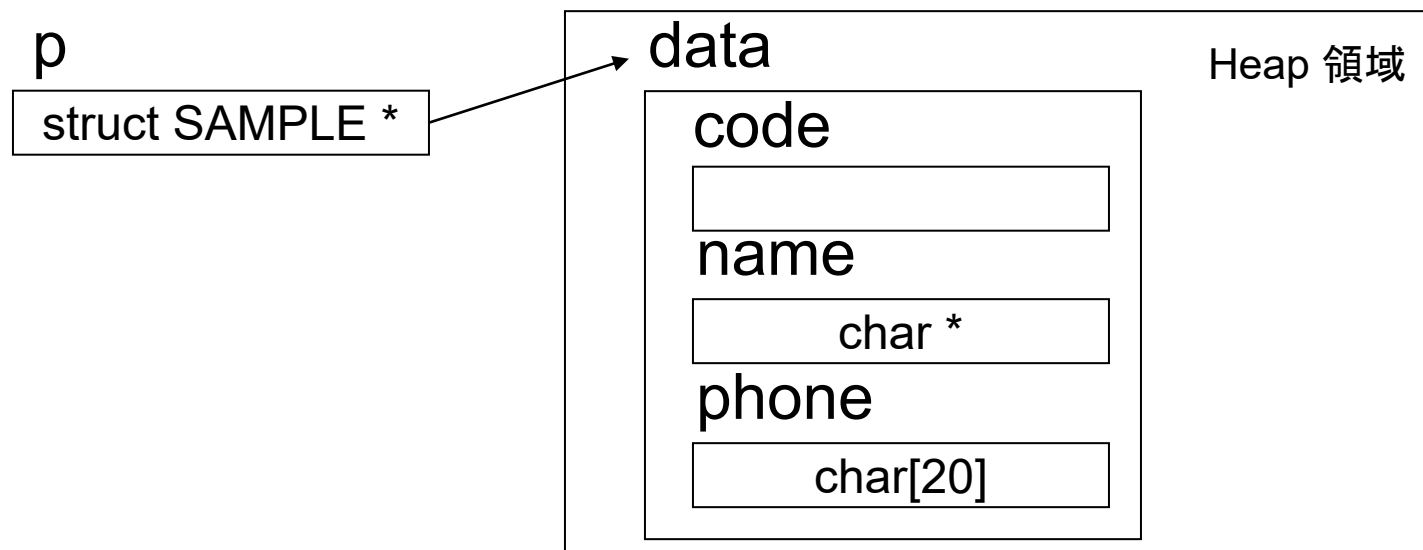


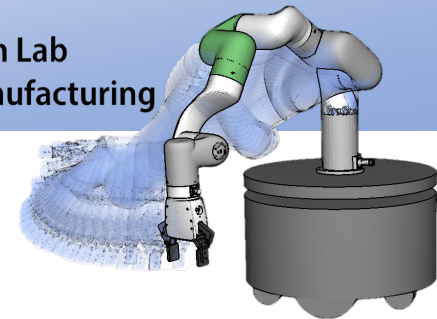
メモリ領域の管理(4)

```
p = (struct SAMPLE *)malloc(sizeof(struct SAMPLE));
```

C言語は型に厳密なので、型が合致しないと代入できない。
p の型は struct SAMPLE * なので、それに型変換（キャスト）する

malloc関数の戻り値は、heap 領域確保したデータ領域の先頭アドレス





メモリ領域の管理(5)

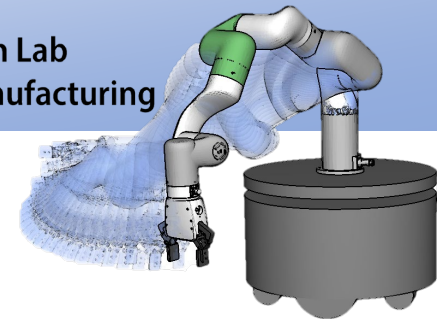
free関数

外部仕様 : `void free(void *ptr)`

`void *ptr; /* 解放する領域の先頭アドレス */`

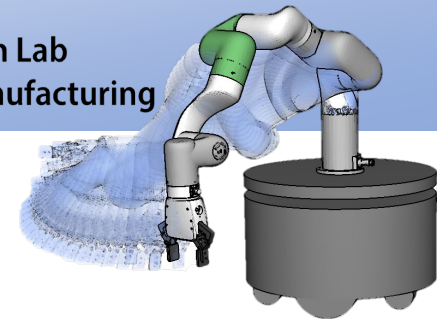
内部仕様 : `ptr` で指定した領域を解放する. なお, `ptr` は

`malloc`関数で確保された領域でなければならない.
`free`関数で解放したあとの領域の値は保証されず,
`malloc`関数で再利用される.



総称ポインタ void *

- どんな型のアドレスでも保管できる特殊なポインタ.
- free 関数のように, 渡されるポインタ型が不明な場合, ライブラリ型では void * が利用される
- マクロNULLは, ポインタで指し示すアドレスがないことを意味する. void*型で値は0である.
 - # define NULL (void*) 0



strdup関数の実装

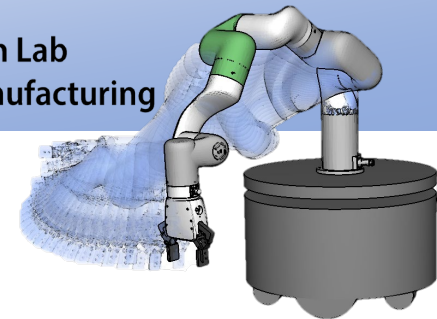
```
char *strdup(char *s)
{
    char *p;
    p = (char *) malloc( strlen(s) + 1 );
    if (p != NULL)
        strcpy(p, s);
    return p;
}
```

文字列の長さを返す関数

(終端記号含めない)

```
char *pCopy = strdup("ABCDE");
puts(pCopy); // ABCDE
free(pCopy);
```

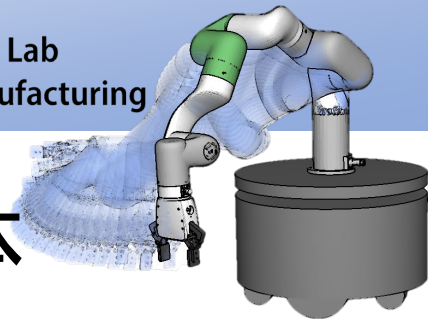
freeでメモリ開放できる



例題 有理数を表す構造体

- 分数（有理数）を表す構造体 `rational` を定義せよ。ここで、分母、分子は整数とする。この構造体を用いて、分数の四則演算（たし算、引き算、かけ算、割り算）をする関数をそれぞれ定義せよ。

```
struct rational {  
    int numerator; /* 分子 */  
    int denominator; /* 分母 */  
};  
  
void add( struct rational* a, struct rational* b, struct rational *c);  
void sub( struct rational* a, struct rational* b, struct rational *c);  
void mul( struct rational* a, struct rational* b, struct rational *c);  
void div( struct rational* a, struct rational* b, struct rational *c);
```



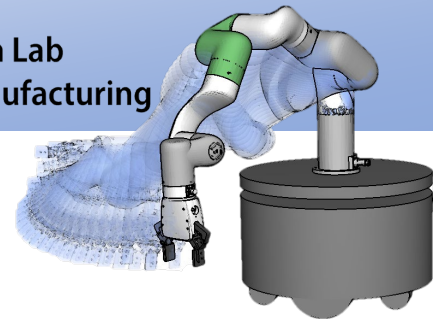
演習課題 有理数を表す構造体

- 足し算の実装

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bd}$$

```
void add( struct rational* a, struct rational* b, struct rational *c)
{
    c->denominator = a->denominator * b->denominator;
    c->numerator = a->numerator * b->denominator
                  + b->numerator * a->denominator;
}
```

実際には、常に分子分母の最大公約数を求めて通分しておくのが望ましい → 最大公約数を求めるアルゴリズムとしてユークリッドの互除法



typedef

- typedef : 既存の型に新しい名前を付ける命令

typedef 既存のデータ型 新しい名前;

例)

```
typedef unsigned int Uint;
```

```
unsigned int a;
```

と

```
Uint a;
```

は同じ意味になる.

応用例

typedefを使って構造体の宣言を簡略
今まで構造体の宣言

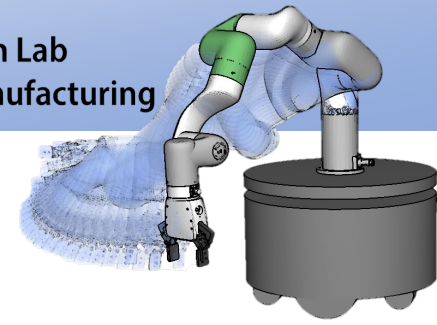
struct 構造体のタグ名 変数名

簡略名を定義

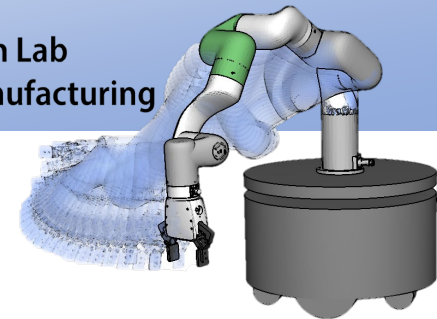
typedef struct 構造体のタグ名 簡略名

簡略された構造体の宣言

簡略名 変数名

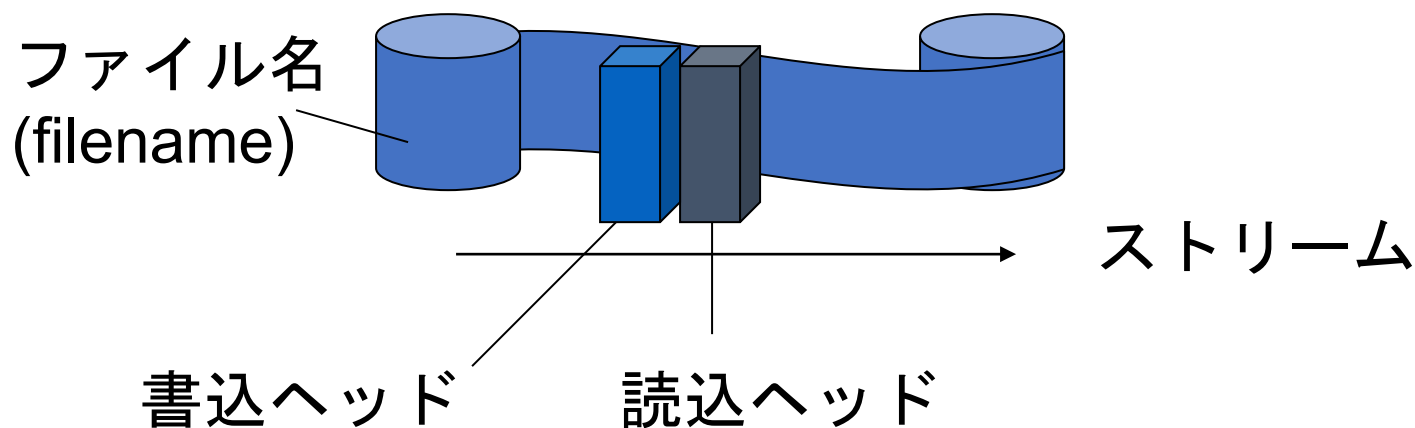


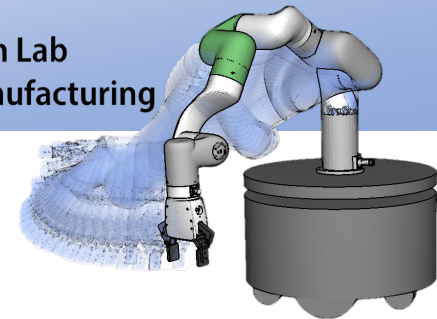
C言語：ファイル



ファイル(FILE)

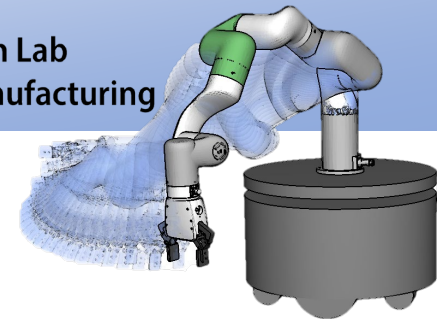
- 抽象データ型(Abstract Data Type)のひとつ





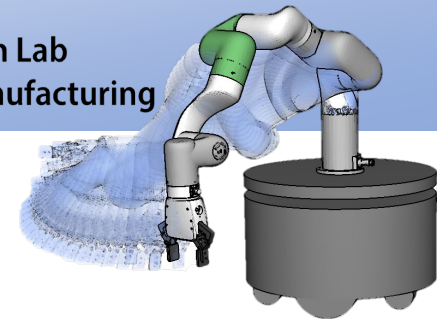
ファイル(FILE)

- ADT FILE に可能な操作
 - ファイルをセットする(Set)
 - ファイルを外す(Unset)
 - 頭出し (Rewind)
 - 読み込み(Read/Get)
 - 書き込み(Write/Put)
 - 現在のヘッド位置を知る(Tell)
 - ヘッド移動(Seek)
 - ランダムアクセス可能なファイルのみ



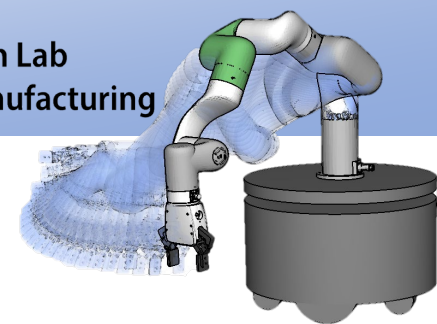
C言語のファイル

- FILEをデータ型としては持っていない
- 代わりにADTのFILE を提供するライブラリがある (stdio.h)
 - fopen ファイルをセットする関数
 - fclose ファイルを外す関数
 - rewind 巻き戻し（ファイル先頭へ移動）
 - fread/getc データの読み込み関数
 - fwrite/putc データの書き込み関数
 - ftell ヘッド位置を知る関数
 - fseek ヘッドを移動する関数



C言語のファイル操作

- ファイルは，最初にファイル名を指定し，`fopen` 関数を呼び出すことでファイルをセットしないと使えない.
- セットすると，`fread/fwrite/fget/fput` で読み書きすることができる
- ファイルは，使い終わったら `fclose`関数を呼び出してファイルを外す.
 - ファイルは計算機資源を消費するので使い終わったら速やかに外す

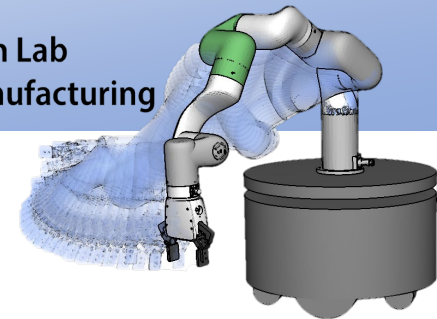


ファイル入出力例

```
#include <stdio.h>    /* FILE を利用するために必要 */

int main(void) {
    FILE *fp; /* FILE *型の変数を宣言 */
    int c;

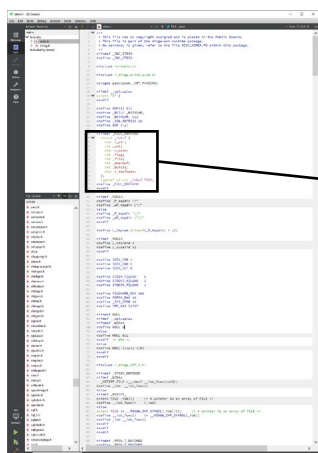
    fp = fopen( "DataFile", "r" ); /* "DataFile"を読み込み用("r")にオープン */
    if (fp == NULL) { /* オープンできないときは NULL が返る */
        printf( "Can't open DataFile!¥n" );
        exit(1);
    }
    while ((c = getc(fp)) != EOF) /* 1 文字読み込む. EOFでなければループ */
        print( "%c", c ); /* 画面に 1 文字出力 */
    fclose( fp ); /* 使い終わったらクローズする */
    return 0;
}
```



FILE *fpの中身

- FILE *fpは，構造体FILEへのポインタ

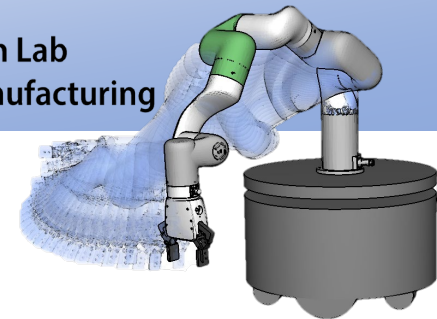
stdio.h



```

25 #ifndef _FILE_DEFINED
26   struct _iobuf {
27     char *_ptr;
28     int _cnt;
29     char *_base;
30     int _flag;
31     int _file;
32     int _charbuf;
33     int _bufsiz;
34     char *_tmpfname;
35   };
36   typedef struct _iobuf FILE;
37 #define _FILE_DEFINED
38 #endif
  
```

- NULL : 値のないポインタを意味する. void* 型で値は0
- EOF (End of file) : ファイル末尾を表す (-1)



まとめ

- C言語のデータ型
 - 情報処理演習に勉強していなかったもの
 - ポインタ・文字列・構造体・ファイル
 - 左辺値、右辺値、代入、参照、前進、後進、等価
 - コピー、代入、連結、比較、strlen/sizeof
 - 宣言、アクセス、malloc/free、heap/stack、typedef
 - CのFILEの宣言、fopen/fclose、読み込みヘッド、書き込みヘッド